

UNIVERSIDAD EUROPEA DE VALENCIA
Grado en Física

Subject: Introduction to artificial intelligence

Classifying phases in the Ising Model using ensemble learning

Author:
Pau Gallego Cisteró

Date:
April 9, 2026

CONTENTS

1	Summary and objectives	2
2	Principles	2
2.1	Metropolis Monte Carlo Simulation	2
2.2	Parallel Ensembles: Bagging and Random Forest	2
2.3	Feature Importance and Physical Interpretation	3
3	Analysis	3
3.1	Bagging	3
3.2	Random Forest	4
4	Questions	4
5	Conclusions	5
6	Code	5
6.1	Dataset generation	5
6.2	Bagging	7
6.3	Random Forest	9
7	AI usage	11

1. SUMMARY AND OBJECTIVES

This report explores the implementation of ensemble learning techniques applied to the classification of physical phase transitions in the 2D Ising model. By utilizing a dataset generated from scratch through the Metropolis Monte Carlo algorithm, which produces lattice configurations of $+1/-1$ spins, the study investigates how collective decision-making can accurately distinguish between ordered and disordered physical states. The analysis focuses on the practical application of ensemble architectures to process stochastic spin lattices, examining how these methods improve the detection of emergent patterns and physical symmetries compared to individual decision trees.

The primary objective of this activity is to develop a computational physics pipeline that integrates data generation with the comparative evaluation of ensemble strategies. Beyond calculating standard metrics such as accuracy and confusion matrices, the study aims to interpret the underlying physical logic of the models by visualizing feature importance heatmaps to identify influential spin sites within the lattice. Ultimately, this analysis seeks to provide a clear intuition on how machine learning identifies statistical mechanical properties, determining how variations in lattice size and proximity to the critical temperature (T_c) influence the reliability and robustness of automated phase classification.

2. PRINCIPLES

The theoretical framework of this activity is based on the application of ensemble learning to statistical mechanics. The core principle posits that aggregating multiple "weak" models—specifically decision trees—can result in a "strong" classifier capable of identifying physical phase transitions. In the context of the Ising model, ensemble methods are used to mitigate the variance associated with stochastic Monte Carlo configurations, improving the model's ability to generalize across different temperature regimes.

2.1. Metropolis Monte Carlo Simulation

To generate a dataset for classification, we rely on the *Metropolis algorithm*, a Markov Chain Monte Carlo (MCMC) method. The principle involves simulating a lattice of $N \times N$ spins where each site is $+1$ or -1 . By iteratively flipping spins based on the Boltzmann distribution, the system reaches equilibrium at a given temperature T . This allows for the generation of two distinct classes of data:

- **Ordered Phase:** At low temperatures ($T < T_c$), where spins tend to align.
- **Disordered Phase:** At high temperatures ($T > T_c$), where thermal fluctuations dominate and spins are randomly oriented.

2.2. Parallel Ensembles: Bagging and Random Forest

The study primarily investigates parallel ensemble techniques designed to reduce model variance through *aggregation*:

- **Bagging (Bootstrap Aggregating):** This principle involves training multiple decision trees independently on different "bootstrapped" subsets of the spin configurations. The final classification is determined by a majority vote, which smooths out errors caused by atypical stochastic configurations in individual samples.
- **Random Forest:** This algorithm builds upon Bagging by introducing "feature randomness." By selecting only a random subset of spin sites for each split, the model ensures that the trees are decorrelated. This is investigated to determine if the ensemble can identify global phase properties rather than becoming over-reliant on a few specific lattice coordinates.

2.3. Feature Importance and Physical Interpretation

A key investigative principle of ensemble learning is the ability to extract *Feature Importance*. By analyzing which spin sites (features) the Random Forest relies on most to make a classification, we can generate heatmaps. Physically, this allows us to interpret whether the model has "learned" the concept of magnetic domains or long-range order, identifying which regions of the lattice are most influential in determining the overall physical phase of the system.

3. ANALYSIS

3.1. Bagging

The Metropolis algorithm generates data that is very noisy, especially at high temperatures where spins flip randomly. By training multiple independent trees on different random subsets of the dataset and averaging their votes, the Bagging model successfully reduced this variance. As a result, the model achieved high accuracy on the test set, indicating that it learned to distinguish between the solid clusters of the ordered phase and the static-like noise of the disordered phase without obvious signs of under-fitting.

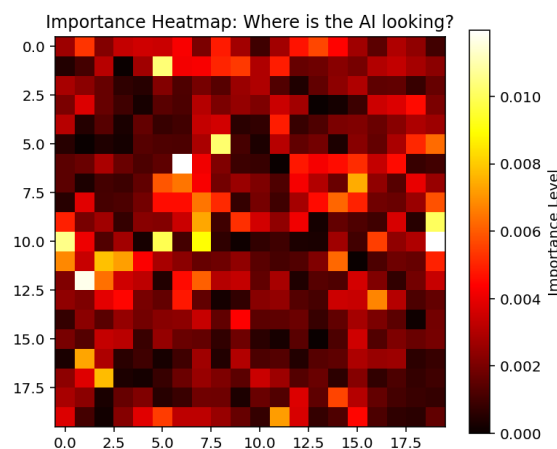


Figure 1: Bagging HeatMap

When the averaged feature importance from all the trees was plotted (Figure 1), the heatmap showed that the model's attention is distributed across the entire 20×20 grid rather than focusing on one specific corner, because the ensemble is looking at the global state of the lattice to make its decision.

3.2. Random Forest

The Random Forest classifier was trained on the same dataset and performed similarly well, but its internal strategy is slightly different. While Bagging gives every tree access to all 400 spin sites, Random Forest forces each tree to make decisions using only a random subset of those sites. If a few spins happen to align by pure chance in a hot lattice, a standard decision tree might get tricked and overfit to that specific spot, Random Forest prevents this by ensuring the trees are not correlated.

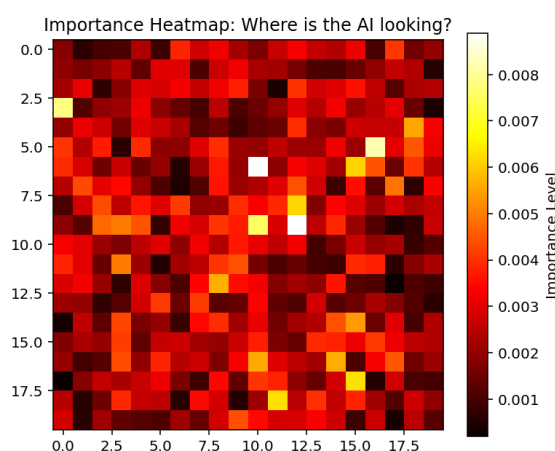


Figure 2: Random Forest HeatMap

When we look at the feature importance heatmap for Random Forest (Figure 2), we can see that it looks practically identical to the Bagging heatmap. This visual similarity makes perfect sense, even though Random Forest uses a slightly different internal trick, both models are trying to solve the exact same physical problem. Because the Ising model is symmetric, no single pixel is truly more important than another. The scattered "hot spots" we see in both heatmaps are just random noise from our limited dataset.

The fact that both heatmaps look like random static proves that both models correctly rely on the overall spatial correlation of the grid, rather than memorizing specific pixels.

4. QUESTIONS

Are some spin sites more influential than others?

The importance heatmap generated by the Random Forest model shows that certain coordinates in the 20×20 lattice have higher weights than others. However, in physics,

every spot in the grid is exactly the same; the reason we see brighter spots is just because our grid is small and the simulation has random noise. If we used a giant grid and ran the code for a long time, theoretically, the heatmap should look completely flat and equal.

What physical interpretation could be drawn from the importance heatmap?

The importance heatmap is a visual tool to check if the AI is actually learning the physics of the system or if it is just finding a mathematical shortcut. If the heatmap were highly concentrated in one small area, it would imply the model is overfitting to local areas, meaning it would be cheating by only looking at a tiny patch of the sample. Because the importance is widely distributed across the entire grid in our plots, the physical interpretation is that the model has successfully recognized "long-range order", understanding that a physical phase transition (like a magnet cooling down and aligning) is a macroscopic event that involves the collective behavior of the entire lattice.

5. CONCLUSIONS

Both models successfully navigated the stochastic noise inherent in the Metropolis Monte Carlo simulations, achieving high classification accuracy between the ordered and disordered states. The aggregation mechanism of these models allowed them to establish robust decision boundaries without succumbing to overfitting. Furthermore, the extraction and visualization of feature importance provided critical physical insights into how the algorithms process the data. The widely distributed, uniform nature of the importance heatmaps confirmed that the artificial intelligence did not inappropriately memorize specific lattice coordinates, which would violate the translational symmetry of the system. This shows that both models not only predict, but also successfully interpret the physical dataset.

6. CODE

The following section details the code that was created for all this report.

6.1. Dataset generation

The following set given illustrates how the initial dataset was created and tampered according to the directives given following a Metropolis Montecarlo.

```
1 #-----  
2 # Dataset generation  
3 #-----  
4  
5 import numpy as np  
6 import matplotlib.pyplot as plt  
7 from sklearn.model_selection import train_test_split
```

```
8 from sklearn.ensemble import RandomForestClassifier
9 from sklearn.metrics import accuracy_score
10
11 lattice = 20
12 cold_temp = 1.5 # temp (Ordered)
13 hot_temp = 4.0 # temp (Disordered)
14 samples = 200
15 steps = 15000
16
17 # Esto aplica Condiciones de Contorno Periodicas
18 def get_neighbors_sum(grid, r, c, n):
19
20     above = grid[(r - 1) % n, c]
21     below = grid[(r + 1) % n, c]
22     left = grid[r, (c - 1) % n]
23     right = grid[r, (c + 1) % n]
24
25     return above + below + left + right
26
27 # MONTECARLO
28 def should_we_flip(current_spin, neighbors_sum, temperature):
29
30     # how much energy change this flip would cause
31     # If the spin matches neighbors, energy is low. If it flips away,
32     # energy rises.
33     energy_change = 2 * current_spin * neighbors_sum
34
35     # Rule A: If flipping REDUCES energy (de <= 0), the system loves it.
36     # Always flip.
37     if energy_change <= 0:
38         return True
39
40     # Rule B: If flipping INCREASES energy, we usually don't do it.
41     # But heat allows for 'unstable' moves. We roll a dice.
42     else:
43         # Probability of flipping even though it's high energy:
44         chance_of_flipping = np.exp(-energy_change / temperature)
45
46         # Roll a random number between 0 and 1
47         roll = np.random.random()
48
49         if roll < chance_of_flipping:
50             return True # Heat was high enough to cause the flip
51         else:
52             return False # Stay as you are.
53
54 def create_single_grid(n, temp, steps):
55     # Start with a random mix of +1 and -1
56     grid = np.random.choice([1, -1], size=(n, n))
57
58     for _ in range(steps):
59         # Pick a random spot in the grid
60         r, c = np.random.randint(0, n, size=2)
61         # randit genera nums enteros al azar
62
63         # Get context
64         myself = grid[r, c]
```

```
63     others = get_neighbors_sum(grid, r, c, n)
64
65     # Make the decision
66     if should_we_flip(myself, others, temp):
67         grid[r, c] *= -1 # Flip from 1 to -1 or vice versa
68
69     return grid
70
71 # --- DATA GENERATION ---
72 X = [] # This will hold the flattened grids (the 'pictures')
73 y = [] # This will hold the labels (0 for Cold, 1 for Hot)
74
75 for _ in range(samples):
76     # Make a cold grid
77     cold_grid = create_single_grid(latice, cold_temp, steps)
78     X.append(cold_grid.flatten()) # Flatten 20x20 into a list of 400
79     y.append(0)
80
81     # Make a hot grid
82     hot_grid = create_single_grid(latice, hot_temp, steps)
83     X.append(hot_grid.flatten())
84     y.append(1)
85
86 X = np.array(X)
87 y = np.array(y)
```

6.2. Bagging

Please note that the Bagging part of coding was implemented AFTER the Random Forest but it's presented earlier as it is detailed on the guide for the present memoire.

```
1 #-----
2 # Bagging Classifier
3 #-----
4
5 import numpy as np
6 import matplotlib.pyplot as plt
7 from sklearn.model_selection import train_test_split
8 from sklearn.ensemble import BaggingClassifier
9 from sklearn.tree import DecisionTreeClassifier
10 from sklearn.metrics import accuracy_score
11
12 latice = 20
13 cold_temp = 1.5 # temp (Ordered)
14 hot_temp = 4.0 # temp (Disordered)
15 samples = 200
16 steps = 15000
17
18 def get_neighbors_sum(grid, r, c, n):
19
20     above = grid[(r - 1) % n, c]
21     below = grid[(r + 1) % n, c]
```

```

22     left  = grid[r, (c - 1) % n]
23     right = grid[r, (c + 1) % n]
24
25     return above + below + left + right
26
27 def should_we_flip(current_spin, neighbors_sum, temperature):
28
29     # how much energy change this flip would cause
30     # If the spin matches neighbors, energy is low. If it flips away,
    energy rises.
31     energy_change = 2 * current_spin * neighbors_sum
32
33     # Rule A: If flipping REDUCES energy (de <= 0), the system loves it.
    Always flip.
34     if energy_change <= 0:
35         return True
36
37     # Rule B: If flipping INCREASES energy, we usually don't do it.
    # But heat allows for 'unstable' moves. We roll a dice.
38     else:
39         # Probability of flipping even though it's high energy:
40         chance_of_flipping = np.exp(-energy_change / temperature)
41
42
43         # Roll a random number between 0 and 1
44         roll = np.random.random()
45
46         if roll < chance_of_flipping:
47             return True # Heat was high enough to cause the flip
48         else:
49             return False # Stay as you are.
50
51 def create_single_grid(n, temp, steps):
52     # Start with a random mix of +1 and -1
53     grid = np.random.choice([1, -1], size=(n, n))
54
55     for _ in range(steps):
56         # Pick a random spot in the grid
57         r, c = np.random.randint(0, n, size=2)
58
59         # Get context
60         myself = grid[r, c]
61         others = get_neighbors_sum(grid, r, c, n)
62
63         # Make the decision
64         if should_we_flip(myself, others, temp):
65             grid[r, c] *= -1 # Flip from 1 to -1 or vice versa
66
67     return grid
68
69 # --- DATA GENERATION ---
70 X = [] # This will hold the flattened grids (the 'pictures')
71 y = [] # This will hold the labels (0 for Cold, 1 for Hot)
72
73 for _ in range(samples):
74     # Make a cold grid
75     cold_grid = create_single_grid(lattice, cold_temp, steps)
76     X.append(cold_grid.flatten()) # Flatten 20x20 into a list of 400

```

```
77     y.append(0)
78
79     # Make a hot grid
80     hot_grid = create_single_grid(latice, hot_temp, steps)
81     X.append(hot_grid.flatten())
82     y.append(1)
83
84 X = np.array(X)
85 y = np.array(y)
86
87 # --- TRAIN/TEST SPLIT ---
88 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
89
90 modelo_bagging = BaggingClassifier(estimator=DecisionTreeClassifier(),
91                                   n_estimators=100, random_state=42)
92
93 modelo_bagging.fit(X_train, y_train)
94
95 predictions = modelo_bagging.predict(X_test)
96 score = accuracy_score(y_test, predictions)
97
98 print(f"\nModel Accuracy: {score * 100}%")
99
100 # --- HEATMAP ---
101 # Promedio importancia arboles
102 importances = np.mean([tree.feature_importances_ for tree in
103                       modelo_bagging.estimators_],
104                       axis=0).reshape(latice, latice)
105
106 plt.figure(figsize=(6, 5))
107 plt.title("Importance Heatmap: Where is the AI looking?")
108 plt.imshow(importances, cmap='hot')
109 plt.colorbar(label='Importance Level')
110 plt.show()
```

6.3. Random Forest

```
1 #-----
2 # Random Forest
3 #-----
4
5 import numpy as np
6 import matplotlib.pyplot as plt
7 from sklearn.model_selection import train_test_split
8 from sklearn.ensemble import RandomForestClassifier
9 from sklearn.metrics import accuracy_score
10
11 latice = 20
12 cold_temp = 1.5 # temp (Ordered)
13 hot_temp = 4.0 # temp (Disordered)
14 samples = 200
15 steps = 15000
16
17 def get_neighbors_sum(grid, r, c, n):
```

```

18     above = grid[(r - 1) % n, c]
19     below = grid[(r + 1) % n, c]
20     left = grid[r, (c - 1) % n]
21     right = grid[r, (c + 1) % n]
22
23
24     return above + below + left + right
25
26 def should_we_flip(current_spin, neighbors_sum, temperature):
27
28     # how much energy change this flip would cause
29     # If the spin matches neighbors, energy is low. If it flips away,
30     # energy rises.
31     energy_change = 2 * current_spin * neighbors_sum
32
33     # Rule A: If flipping REDUCES energy (de <= 0), the system loves it.
34     # Always flip.
35     if energy_change <= 0:
36         return True
37
38     # Rule B: If flipping INCREASES energy, we usually don't do it.
39     # But heat allows for 'unstable' moves. We roll a dice.
40     else:
41         # Probability of flipping even though it's high energy:
42         chance_of_flipping = np.exp(-energy_change / temperature)
43
44         # Roll a random number between 0 and 1
45         roll = np.random.random()
46
47         if roll < chance_of_flipping:
48             return True # Heat was high enough to cause the flip
49         else:
50             return False # Stay as you are.
51
52 def create_single_grid(n, temp, steps):
53     # Start with a random mix of +1 and -1
54     grid = np.random.choice([1, -1], size=(n, n))
55
56     for _ in range(steps):
57         # Pick a random spot in the grid
58         r, c = np.random.randint(0, n, size=2)
59
60         # Get context
61         myself = grid[r, c]
62         others = get_neighbors_sum(grid, r, c, n)
63
64         # Make the decision
65         if should_we_flip(myself, others, temp):
66             grid[r, c] *= -1 # Flip from 1 to -1 or vice versa
67
68     return grid
69
70 # --- DATA GENERATION ---
71 X = [] # This will hold the flattened grids (the 'pictures')
72 y = [] # This will hold the labels (0 for Cold, 1 for Hot)
73
74 for _ in range(samples):

```

```
73     # Make a cold grid
74     cold_grid = create_single_grid(lattice, cold_temp, steps)
75     X.append(cold_grid.flatten()) # Flatten 20x20 into a list of 400
76     y.append(0)
77
78     # Make a hot grid
79     hot_grid = create_single_grid(lattice, hot_temp, steps)
80     X.append(hot_grid.flatten())
81     y.append(1)
82
83 X = np.array(X)
84 y = np.array(y)
85
86 # --- TRAIN/TEST SPLIT ---
87 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
88
89 # Random forest
90 # Think of this as 100 little 'experts' looking at the magnets and
   voting
91 forest = RandomForestClassifier(n_estimators=100)
92 forest.fit(X_train, y_train)
93
94 predictions = forest.predict(X_test)
95 score = accuracy_score(y_test, predictions)
96
97 print(f"\nModel Accuracy: {score * 100}%")
98
99 # --- HEATMAP ---
100 # This shows which parts of the grid the AI found most 'important'
101 importances = forest.feature_importances_.reshape(lattice, lattice)
102
103 plt.figure(figsize=(6, 5))
104 plt.title("Importance Heatmap: Where is the AI looking?")
105 plt.imshow(importances, cmap='hot')
106 plt.colorbar(label='Importance Level')
107 plt.show()
```

7. AI USAGE

AI was implemented in this memoir not only to enhance the text and the register used but also to write sections 1 and 2; as well as heavily assisting with the dataset generation and to debug the HeatMap generation for the Bagging method.